FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-00 Protected Practice Review

Diagnostic hints + changed-retry criteria | open only after genuine attempt

Artifact	C3_012_ENGINEERING_FOUNDATIONS_REVIEW_PACK_PROTECTED_RC1
Status	PROTECTED - RC1
Teaching standard	M01 beginner-first teacher loop; book remains sufficient without video.
Bridge boundary	Course 2B competencies may be assumed; all new engineering concepts still start from first principles.
Brand	Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6 | AMOLED #000

House rule: understand -> follow once -> see expected result -> break/fix -> do a changed version -> validate -> explain -> save evidence. A running command alone never counts as mastery.

Protection rule

Opening this review before saving the matching practice attempt invalidates that attempt as independent evidence. The review intentionally gives diagnosis patterns and targeted hints rather than copy-ready final solutions.

P01 review - Role shift: analyst scripts -> engineering systems

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Starting code before stating input/output grain.

• Treating a plausible number as evidence.

• Writing "rerun script" without saying whether overwrite/replay is safe.

• Making the original developer the only person who knows what to check.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

If the contract feels abstract, write one valid row, one invalid row, one expected output, and one rerun story. Turn those concrete examples into rules.

CHANGED RETRY

New domain: support tickets. Write a fresh contract for a utility that creates daily closed-ticket counts by queue. Do not copy the sales wording.

PASS THE RETRY ONLY IF

• Every requirement can be tested yes/no.

• Input and output grain are explicit.

• At least one independent control would catch a plausible wrong result.

• Failure and rerun behavior are explicit.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain role shift: analyst scripts -> engineering systems in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P02 review - Linux/WSL2 shell, paths and filesystem

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Keeping the whole Linux-heavy project under `/mnt/c` because it looks familiar.

• Using `sudo` whenever permission/path reasoning is unclear.

• Running `rm -rf` before checking `pwd`.

• Confusing `~`, `.`, `..` and `/`.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Use inspection-only commands first: `pwd`, `ls -lah`, `realpath .`, `find . -maxdepth 2 -type f`. If unsure, stop before move/delete.

CHANGED RETRY

Create `~/course3/c3_00/path_transfer/{incoming,processed,archive}` from memory. Copy one file to archive, move a different file to processed, then prove the source/target state.

PASS THE RETRY ONLY IF

• Project path is Linux-native.

• Source file you intended to preserve still exists.

• Final tree matches your plan.

• You can explain every relative path from the directory where the command ran.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain linux/wsl2 shell, paths and filesystem in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P03 review - Shell pipes, redirection and process basics

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Using `>` when you meant `>>` and overwriting evidence.

• Checking `$?` after another command, which replaces the status you wanted.

• Assuming empty output means the command crashed.

• Building a long pipeline before validating each stage.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Start with the smallest stage, save intermediate output, inspect stderr, and record exit codes. Reassemble only after each stage behaves as expected.

CHANGED RETRY

On a changed `service.log`, produce counts by level and a separate `ERROR` artifact without modifying the input. Add one deliberate no-match check and explain its result.

PASS THE RETRY ONLY IF

• Source hash is unchanged.

• Counts reconcile to total lines.

• Filtered artifact contains only the requested label.

• Exit status is captured immediately and interpreted correctly.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain shell pipes, redirection and process basics in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P04 review - Git branches, commits and pull-request workflow

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• `git add .` without reviewing the diff.

• One giant commit mixing code, generated outputs and experiments.

• Tracking `.venv`, `.env`, logs or bytecode.

• Using destructive reset/clean commands before understanding status.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Run `git status`, then `git diff` and `git diff --staged`. Prefer reversible operations. Do not use `reset --hard`/`clean -fd` as reflexes.

CHANGED RETRY

Create `feature/logging-contract` in a fresh small repo, change one logger behavior, add/update one test, make two coherent commits, and draft a PR summary.

PASS THE RETRY ONLY IF

• Branch name and commit history show the intended work.

• No secret/disposable environment is tracked.

• Diff is understood file by file.

• PR summary states what/why/how-tested/risk.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain git branches, commits and pull-request workflow in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P05 review - Python virtual environments and dependency pinning

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Committing `.venv`.

• Copying a venv between machines and calling that reproducible.

• Using global `pip` while a different Python interpreter runs the project.

• Recording dependencies but never testing a clean recreation.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Check `which python`, `python --version`, `python -m pip --version`, and `python -c "import sys; print(sys.executable)"`. If state is confused, preserve source/requirements, recreate only the disposable venv.

CHANGED RETRY

Create a second minimal project from scratch with one dependency/test. Recreate it only from README + dependency file and prove the test passes.

PASS THE RETRY ONLY IF

• Fresh recreation succeeds.

• Active interpreter lives in project `.venv`.

• Dependency instructions are sufficient.

• `.venv` is ignored by Git.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain python virtual environments and dependency pinning in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P06 review - Python modules, packages and project structure

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Creating many files with no responsibility boundary.

• Running CLI parsing at import time.

• Duplicating business logic in CLI and tests.

• Using relative filesystem assumptions inside core logic.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Draw boxes for INPUT/CLI, CORE, OUTPUT. Move one responsibility at a time, run tests after each move, and inspect imports before adding more structure.

CHANGED RETRY

Refactor a different `daily_summary.py` utility so core aggregation is callable independently and the CLI is thin.

PASS THE RETRY ONLY IF

• Core imports without side effects.

• Unit test reaches core without CLI/file-system ceremony where possible.

• CLI delegates rather than duplicates rules.

• Project layout is documented.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain python modules, packages and project structure in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P07 review - Configuration, environment variables and secret boundaries

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Hard-coding Windows paths inside reusable code.

• Logging an API token while debugging.

• Treating `.env.example` as a place for real credentials.

• Having several config sources with undocumented precedence.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Print only non-sensitive resolved configuration, one field at a time. Confirm precedence with three tiny runs before debugging downstream code.

CHANGED RETRY

Add `C3_LOG_LEVEL` with CLI override `--log-level`. Reject unsupported values early and prove precedence on a changed run.

PASS THE RETRY ONLY IF

• Precedence is documented and demonstrated.

• Invalid values fail before processing.

• No secret is tracked/logged.

• Default is safe and predictable.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain configuration, environment variables and secret boundaries in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P08 review - Structured logging and exception handling

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Logging every row and creating noise/PII risk.

• Catching `Exception` everywhere.

• Printing a secret while debugging.

• Writing ERROR but still returning success.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Reproduce with the smallest input, increase log detail deliberately, inspect the first meaningful error, and follow the exception chain rather than adding random try/except blocks.

CHANGED RETRY

On a changed CSV utility, design INFO/WARNING/ERROR events and demonstrate one expected validation failure and one unexpected exception path.

PASS THE RETRY ONLY IF

• Success/failure exit behavior matches logs.

• Errors include safe actionable context.

• Unexpected exceptions keep trace evidence at boundary.

• No secret/PII dump.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain structured logging and exception handling in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P09 review - Unit tests and integration-test boundaries

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Only happy-path tests.

• Tests that depend on execution order or a real desktop file.

• Asserting the same formula used by implementation without independent expected values.

• Calling every test an integration test.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Start from one input and one expected output. Make the function smaller if setup is huge. Use `tmp_path`; inspect the first failure; do not weaken assertions just to get green.

CHANGED RETRY

For a changed file-inspector function, write at least 4 tests including one edge case and one failure contract. Deliberately inject one regression and prove the suite catches it.

PASS THE RETRY ONLY IF

• Deliberate regression causes red.

• Clean code causes green.

• Filesystem tests are isolated.

• Test names communicate behavior.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain unit tests and integration-test boundaries in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P10 review - CLI design and reusable data utilities

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Business logic inside `if __name__ == "__main__"` only.

• No `--help` or ambiguous argument names.

• Changing output path/name every run.

• Returning success after validation failure.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Run `--help`, test one valid invocation, then one intentionally invalid invocation. If behavior is unclear, simplify parser and document exit contract before adding features.

CHANGED RETRY

Build a changed `rowcount` utility CLI with input path and JSON output, one validation error, stable help and a reusable core function.

PASS THE RETRY ONLY IF

• Help is understandable.

• Success and failure exit codes differ.

• Output schema/path are stable.

• Core can be tested without CLI parsing.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain cli design and reusable data utilities in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P11 review - Docker and container mental model

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

• Installing Docker at C3-00 start even though not needed yet.

• Calling an image a running container.

• Writing output only inside an ephemeral container and losing evidence.

• Claiming Docker success without a real command run.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Check `docker version`, Docker Desktop engine state, WSL integration, then a tiny known image. Separate installation/engine/network/build problems rather than changing everything at once.

CHANGED RETRY

Explain and, if available, run a changed containerized `rowcount` CLI with a host-mounted output. If Docker cannot safely run, produce the Dockerfile + exact commands + expected result + blocker evidence.

PASS THE RETRY ONLY IF

• Image vs container explanation correct.

• Client/server engine verified or blocker recorded honestly.

• `--help` works in real container when available.

• Output persistence/mount reasoning is correct.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain docker and container mental model in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.

P12 review - Idempotence, reproducibility and runbooks

FIRST DIAGNOSE YOUR ATTEMPT

Before reading further, write one sentence: what failed or remains uncertain, and which evidence proves that diagnosis.

COMMON FAILURE SIGNALS

- Calling every repeated command idempotent without checking business effect.
- Demanding identical logs when timestamps legitimately differ.
- A runbook that says only "run script.py".
- Recovery instructions that delete state before diagnosis.

HELP LADDER

Level 1: restate the intended contract/result. Level 2: inspect the smallest relevant state/command. Level 3: compare with the lesson mental model. Level 4: reveal only the specific pattern below; do not copy a finished artifact.

TARGETED PATTERN

Restate the intended end state. Compare input identity/config, inspect previous output, run on a copy/test path, then choose replay/overwrite behavior deliberately.

CHANGED RETRY

Changed domain: daily inventory snapshot. Define safe rerun semantics, validate two unchanged reruns, create one failure/recovery drill and write a concise operator runbook.

PASS THE RETRY ONLY IF

- Rerun rule matches the business/output grain.
- Repeated execution does not multiply intended effect.
- Failure recovery preserves known-good state where contract requires.
- Another operator can follow runbook without hidden steps.

EXPLAIN-BACK CHECK

Without the lesson/review open, explain idempotence, reproducibility and runbooks in simple English, one failure mode, and the validation that would catch it.

IF STILL WEAK

Use the targeted remediation card for this competency, then perform another fresh changed micro-retry. Do not repeat the original exact task until memorized.